Trace the dead end.

A replacement developer inherits a checkout integration. Before restarting the Stripe experiment, they need to know why the team stopped and whether the blocker still applies.

The case

In the fictional demo, PR #42 records a regional checkout failure. Issue #17 says the test environment was later fixed. DeadEnd surfaces both excerpts so a reviewer can explain what changed before the next developer acts.

Try the working build

Open an example case. Select the Stripe finding, then inspect its evidence and review the later update. Edit or approve the finding. Open Handover to see the source, next action and approved interpretation.

https://deadend-ai-codersinpune.vercel.app/

Submission identity

Team NIKKUSHRIVASTAVA0. Team leader: Nitanshu Shrivastav. Built for the CodersInPune 2026 challenge on preserving project knowledge through personnel changes.

From source to handover

The investigation follows one technical choice through its source, interpretation, evidence, human review and next action.

01 Import selected records

Paste authorised notes or a public GitHub issue or pull-request URL. Issue comments, review summaries and inline PR comments retain separate source links. The importer reads up to 100 entries per comment or review category.

02 Inspect the evidence

Rules extract possible rejections and blockers. Exact quotation and source-link checks run before approval and again at export. Missing reasoning is marked for context. A named technology appearing in a reversal can trigger a possible-conflict flag.

03 Make a human decision

The CSS 3D workspace supports orbit, node selection and finding selection. Reviewers can change the interpretation while preserving the quote, dispute or reject a claim, reopen it, or approve a public finding.

04 Hand over the next action

The handover contains the approved interpretation, original excerpt, source and suggested recheck. Restricted and rejected records stay out. The reviewer can download the result as Markdown.

What changed since Round 1

Round 1 proposed Next.js, an agent extractor and Supabase. Round 2 extends the supplied v0.6 Node prototype. The public build runs on Vercel and uses the deterministic extractor.

Browser and server

Semantic HTML, JavaScript and CSS perspective drive the evidence graph. The Node API handles GitHub ingestion, extraction, review and export. Vercel sessions carry encrypted, authenticated snapshots in tab storage; reloading retains the case.

Optional model path

A server-side adapter requests structured JSON containing an exact quote and segment index. It rejects excerpts absent from that segment. A model outage leaves rule-based candidates available. This path was tested with fixtures only.

Verification

15 automated tests passed. Coverage includes comment links, quote tampering, imports, API failure, conflicting text, review edits, restricted export, model fallback, encrypted state and private-instance restart recovery.

Public browser checks

An anonymous browser loaded five fictional findings from three sources, inspected the later Stripe update, approved one finding and opened its handover. A real GitHub issue import recovered 34,567 characters and returned zero candidates. Reload preserved review state.

Limits and next round

The public demo accepts synthetic or authorised public records. Its review decisions belong to one browser session.

Current operating limits

The hosted session expires two hours after its last write. There is no team database, per-person identity or private GitHub integration. Related-update matching is heuristic. The optional disk store and shared reviewer password are for a separate single-process installation.

1 Permission-aware team handover

Add individual sign-in, database-backed membership and source-level access rules. Verify that two recipients with different roles receive different authorised excerpts, and log each acknowledgement.

2 Resolve conflicting decisions

Link competing evidence across dates, allow owners to mark a decision superseded and retain the full correction history. Test the new chain against a labelled set of reversals and unrelated mentions.

3 Revalidate when conditions change

Track the specific dependency, policy or environment behind each rejection. When a linked source changes, reopen the finding for review and record the owner and next action.

Development disclosure: the team used AI assistance for implementation, design, testing and documentation. Demo discussions are synthetic.